

Documentación del Proyecto A.R.I.A.

A.R.I.A. (Analizador de Reuniones con Inteligencia Artificial) es un sistema de escritorio desarrollado en C++ utilizando el framework Qt 6. Su objetivo principal es automatizar el procesamiento de transcripciones de reuniones, utilizando IA generativa para extraer participantes, generar actas formales, crear resúmenes ejecutivos y asignar tareas.

1. Funcionalidades Principales

- **Procesamiento Inteligente:** Integración con la API REST de Google Gemini 2.5 Flash para interpretar textos extensos.
 - **Carga Multiformato:** Permite cargar transcripciones arrastrando archivos (Drag & Drop) o seleccionando archivos en formato `.txt` o `.docx`.
 - **Generación Estructurada:** Produce automáticamente:
 - Un resumen ejecutivo breve.
 - Un acta formal detallada.
 - Una lista estructurada de tareas asignadas por participante (con descripción y plazo).
 - **Exportación a PDF:** Permite descargar el resultado completo como un archivo PDF formateado mediante `QPrinter` y `QTextDocument`.
 - **Persistencia Híbrida:**
 - **Local (Offline):** Almacenamiento en base de datos `SQLite` para disponer de un historial inmediato en la PC del usuario.
 - **Remoto (Nube):** Sincronización automática de cada análisis con una base de datos `MySQL` remota (VPS) a través de una API REST en `PHP`.
-

2. Arquitectura del Sistema

El sistema respeta una arquitectura en capas que asegura una fuerte separación de responsabilidades:

Capa de Presentación (Vista)

- **MainWindow** : Clase principal encargada de la Interfaz Gráfica de Usuario (GUI). Gestiona el `QTabWidget` y la captura de eventos (clicks, drag & drop).
 - *Regla de diseño*: No realiza procesamiento de lógica de negocio. Solo delega tareas a la capa lógica y muestra resultados.

Capa de Lógica de Negocio (Controlador / Servicios)

- **AlgoritmoIA (Interfaz Abstracta)**: Define el contrato que cualquier IA debe cumplir (métodos `analizar`, `construirPrompt`, `parsearRespuesta`).
- **AnalizadorGemini** : Implementación concreta de la IA. Gestiona el armado del JSON y la comunicación HTTP con Google mediante `QNetworkAccessManager` .
- **ExportadorPDF** : Servicio dedicado de forma exclusiva a convertir estructuras de datos en archivos PDF estéticos.

Capa de Datos (Modelo)

- **DataManager** : Singleton central que gestiona el estado de la sesión y la comunicación con las bases de datos (tanto SQLite local como el API de MySQL remota).
-

3. Conceptos de POO Aplicados

El proyecto fue diseñado para demostrar el uso exhaustivo de los pilares de la Programación Orientada a Objetos:

Herencia y Clases Abstractas

- Se diseñó la clase abstracta `AlgoritmoIA` . Al contener métodos virtuales puros (como `virtual void analizar() = 0;`), esta clase no se puede instanciar directamente, sino que sirve como plantilla estricta.
- `AnalizadorGemini` hereda de esta clase base y proporciona el código funcional para la API de Google. Si mañana se deseara usar OpenAI, bastaría con crear un `AnalizadorOpenAI` que herede de la misma base, garantizando la extensibilidad.

Polimorfismo

- En `MainWindow` , el motor de IA se almacena como un puntero genérico a la clase base: `AlgoritmoIA *m_algoritmo;` .

- La interfaz gráfica **no sabe** qué IA está ejecutando, simplemente llama a `m_algoritmo->analizar()`. En tiempo de ejecución (runtime), C++ resuelve dinámicamente la llamada hacia la implementación real en `AnalizadorGemini`. Esto permite que la vista esté totalmente desacoplada de la implementación del algoritmo.

Static y Singleton

- **Singleton en `DataManager`**: Para garantizar que no existan múltiples instancias desincronizadas del historial, `DataManager` tiene constructores privados y se accede mediante un método estático `instancia()`.
 - **Atributos Static**: Se usan en `DataManager` para llevar un `m_contadorAnalisis` global, y en `AnalizadorGemini` para almacenar constantes compartidas como `MODELO` o `ENDPOINT_BASE`.
-

4. Persistencia de Datos (Esquema Dual)

Para cumplir con los requerimientos locales y remotos, el sistema implementa una escritura dual que se ejecuta tras completarse un análisis exitoso:

1. Persistencia Local (SQLite):

- La app utiliza `QSqlDatabase` con el driver `QSQLITE`.
- El archivo de la base de datos se crea físicamente en el directorio raíz del proyecto fuente (`PRO_DIR + "/historial.sqlite"`).
- Garantiza que el usuario tenga un registro histórico personal aunque no haya conexión a Internet.

2. Persistencia Remota (MySQL en VPS):

- Una vez que el análisis termina y se guarda localmente, `DataManager` llama a su método `enviarAlServidor()`.
 - Se empaquetan los datos y se envían vía `HTTP POST` hacia una API REST (`api.php`) ubicada en el VPS, que los inserta en una tabla de MySQL (`poo_db.analisis`).
 - *Nota Arquitectónica*: Si bien actualmente esta base remota se utiliza para respaldar los resultados del análisis y demostrar la conectividad a la nube, el objetivo principal y original de esta infraestructura remota es soportar un sistema de **Login y Autenticación de Usuarios** (el cual se encuentra actualmente inactivo/en desarrollo en la rama principal).
-

5. Flujo Completo de Ejecución

1. **Carga:** El usuario carga un `.txt` o `.docx` en la UI e ingresa el tema de la reunión.
2. **Invocación Polimórfica:** `MainWindow` recolecta el texto y la API key, y ejecuta `m_algoritmo->analizar()`.
3. **Procesamiento:** `AnalizadorGemini` empaqueta la solicitud y la envía a la API de Google mediante un request asíncrono.
4. **Parseo:** Al recibir el string JSON de Google, el método sobreescrito `parsearRespuesta()` desarma el JSON y lo carga en una estructura limpia `ResultadoAnálisis` de C++.
5. **Persistencia:** La estructura se pasa a `DataManager::instancia().registrarAnálisis()`, que la inserta en SQLite, y a continuación, en MySQL a través de `enviarAlServidor()`.
6. **Visualización:** `AnalizadorGemini` emite la señal `análisisCompleto`. `MainWindow` la intercepta (vía un `SLOT`) y actualiza los campos de texto en pantalla.
7. **Exportación (Opcional):** El usuario puede hacer clic en "Exportar PDF" y `ExportadorPDF` transformará la información en un documento.